

BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: Parker Catena, Regina Velasco

ID: pdc76, rv297

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```

Activating project at `~/BEE 4750 /HW 4/hw4-parker-regina`
  Installed ArgCheck      v2.5.0
  Installed MarkdownTables v1.1.0
  Installed DisplayAs     v0.1.6
  Installed StatsBase     v0.34.7
  Installed BenchmarkTools v1.6.3
  Installed DataFrames    v1.8.1
  Installed PrettyTables  v3.1.0
  Installed JuMP          v1.29.2
Precompiling project...
  520.5 ms StaticArraysCore
  683.0 ms ArgCheck
  546.0 ms CommonSubexpressions
  691.1 ms DisplayAs
  714.1 ms StructUtils
  799.4 ms StructTypes
 1027.2 ms IrrationalConstants
  514.0 ms LoggingExtras
  533.7 ms Compat
  681.6 ms libaom_jll
  598.8 ms Expat_jll
  727.5 ms OpenSSL_jll
  657.6 ms CodecBzip2
  889.7 ms HiGHS_jll
  985.1 ms DiffResults
 1190.4 ms StructUtils → StructUtilsTablesExt
 1743.3 ms MarkdownTables
 1357.2 ms LogExpFunctions
 1204.6 ms Compat → CompatLinearAlgebraExt
 1149.5 ms Fontconfig_jll
 3392.4 ms JSON
 5261.8 ms MutableArithmetics
 2139.2 ms OpenSSL
 2764.4 ms StatsBase
 2449.9 ms SpecialFunctions
 1169.2 ms Cairo_jll
 2276.3 ms Qt6Base_jll
 1054.9 ms DiffRules
 2175.8 ms BenchmarkTools
 1431.1 ms ColorVectorSpace → SpecialFunctionsExt
 1209.4 ms HarfBuzz_jll
  7205.1 ms JSON3
 1361.3 ms Qt6ShaderTools_jll

```

```

1272.1 ms    libass_jll
1276.2 ms    Pango_jll
1665.6 ms    Qt6Declarative_jll
 393.9 ms    Qt6Wayland_jll
1614.9 ms    FFMPEG_jll
3972.9 ms    ForwardDiff
 960.1 ms    FFMPEG
1387.7 ms    GR_jll
13771.9 ms   HTTP
2843.4 ms    GR
24502.1 ms   PrettyTables
31447.0 ms   MathOptInterface
2015.6 ms    MathOptIIS
8517.1 ms    HiGHS
31780.1 ms   DataFrames
14475.4 ms   JuMP
2228.3 ms    Latexify → DataFramesExt
44736.9 ms   Plots
51 dependencies successfully precompiled in 69 seconds. 171 already precompiled.

```

```

using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables

```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned}
 & \max_{x,y} && 3x + 2y \\
 & \text{subject to:} && x + 4y \leq 16 \\
 & && x - 2y \leq -1 \\
 & && 0 \leq x \leq 4 \\
 & && 1 \leq y \leq 4.
 \end{aligned}$$

```

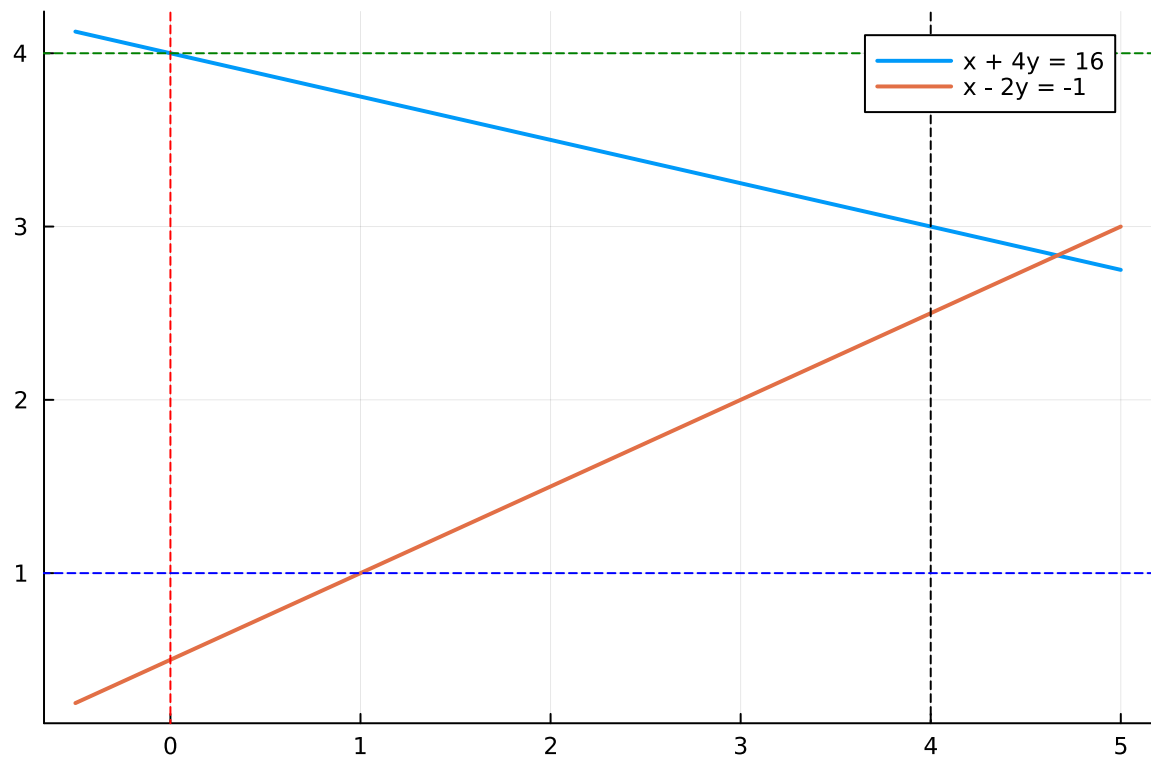
#Set x range for plotting
x = -0.5:0.01:5.0

#Constraint functions
y1(x) = (16 - x) / 4
y2(x) = (x + 1) / 2

#Plot constraint functions across all x values
plt = plot(x, y1.(x), lw=2, label="x + 4y = 16")
plot!(plt, x, y2.(x), lw=2, label="x - 2y = -1")

#Draw the vertical and horizontal lines representing
#the bounds for constraints
vline!([0], linestyle=:dash, color=:red, label="")
vline!([4], linestyle=:dash, color=:black, label="")
hline!([1], linestyle=:dash, color=:blue, label="")
hline!([4], linestyle=:dash, color=:green, label="")

```



Using the above graph of constraints, we know that the maximum value must occur at an intersection of the boundaries. We can use these intersections to check the values for the

maximum of the function $3x + 2y$. The feasible intersections, as determined by are as follows:

$$(0, 1), (0, 4), (1, 1), (4, \frac{5}{2}), (4, 3)$$

We can then check these pairs with the function $3x + 2y$:

$$3(0) + 2(1) = 2, \quad (0, 1)$$

$$3(0) + 2(4) = 8, \quad (0, 4)$$

$$3(1) + 2(1) = 5, \quad (1, 1)$$

$$3(4) + 2(2.5) = 17, \quad (4, 2.5)$$

$$3(4) + 2(3) = 18, \quad (4, 3)$$

Therefore, we can see that at $(4, 3)$ we achieve the maximum value of $3x + 2y = 18$.

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Soybeans	350	0.36

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear.**

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

SOLUTION 2.2

The linear program recommends:

Soybeans: 69.06 ha total

13.81 ha at 0 kg/ha

55.25 ha at 1 kg/ha

Wheat: 22.48 ha total

6.74 ha at 0 kg/ha

15.73 ha at 1 kg/ha

Corn: 38.46 ha total

26.92 ha at 0 kg/ha

11.54 ha at 2 kg/ha

This will yield a maximized profit of \$116,741 while remaining within the land constraint of 130 ha limit.

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

Problem 2 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```
#Problem 2.1
using JuMP # optimization modeling syntax
using HiGHS # optimization solver

crop_model = Model(HiGHS.Optimizer) # initialize model object
@variable(crop_model, s[0:2] >= 0) # non-neg constraints for soy
@variable(crop_model, w[0:2] >= 0) # non-neg constraints for wheat
@variable(crop_model, c[0:2] >= 0) # non-neg constraints for corn

# index: 1 = soy, 2 = wheat, 3 = corn

# set up variables:
# selling prices ($/kg)
ps, pw, pc = 0.36, 0.27, 0.22
# production cost ($/ha)
cs, cw, cc = 350, 280, 390
# pesticide cost ($ per kg/ha applied)
pest = 70
# yields (kg/ha)
ys0, ys1, ys2 = 2900, 3800, 4400
yw0, yw1, yw2 = 3500, 4100, 4200
yc0, yc1, yc2 = 5900, 6700, 7900
# average rate caps (kg/ha)
Ls, Lw, Lc = 0.8, 0.7, 0.6
# land & demand
land_cap = 130
demand = 250000 #for each crop
```

```

# OBJ: maximize profits over 130 total ha while remaining in regulatory
# compliance if demand for each crop (which is the maximum the market
# would buy) this year is 250,000 kg
# Objective = revenue - production costs - pesticide costs
@objective(crop_model, Max,
    # revenue
    + ps*(ys0*s[0] + ys1*s[1] + ys2*s[2])
    + pw*(yw0*w[0] + yw1*w[1] + yw2*w[2])
    + pc*(yc0*c[0] + yc1*c[1] + yc2*c[2])
    # production costs
    - cs*sum(s[i] for i in 0:2)
    - cw*sum(w[i] for i in 0:2)
    - cc*sum(c[i] for i in 0:2)
    # pesticide costs (rate * pest * hectares)
    - pest*( 0*s[0] + 1*s[1] + 2*s[2]
            + 0*w[0] + 1*w[1] + 2*w[2]
            + 0*c[0] + 1*c[1] + 2*c[2])
)

# Constraints

# Land
@constraint(crop_model, land, sum(s[i] for i in 0:2) + sum(w[i] for i in 0:2) + sum(c[i] for i in 0:2) == 130)

# Demand caps (kg)
@constraint(crop_model, ds, ys0*s[0] + ys1*s[1] + ys2*s[2] <= demand) # soy
@constraint(crop_model, dw, yw0*w[0] + yw1*w[1] + yw2*w[2] <= demand) # wheat
@constraint(crop_model, dc, yc0*c[0] + yc1*c[1] + yc2*c[2] <= demand) # corn

# Average pesticide caps: sum(rate*x) L * sum(x)
@constraint(crop_model, pest_s, (0*s[0] + 1*s[1] + 2*s[2]) <= Ls*(sum(s[i] for i in 0:2)))
@constraint(crop_model, pest_w, (0*w[0] + 1*w[1] + 2*w[2]) <= Lw*(sum(w[i] for i in 0:2)))
@constraint(crop_model, pest_c, (0*c[0] + 1*c[1] + 2*c[2]) <= Lc*(sum(c[i] for i in 0:2)))

print(crop_model)

```

```

Max 694 s[0] + 948 s[1] + 1094 s[2] + 665.00000000000001 w[0] + 757 w[1] + 714 w[2] + 908 c[0]
Subject to
    land : s[0] + s[1] + s[2] + w[0] + w[1] + w[2] + c[0] + c[1] + c[2]    130
    ds : 2900 s[0] + 3800 s[1] + 4400 s[2]    250000

```



```

dw : 3500 w[0] + 4100 w[1] + 4200 w[2]    250000
dc : 5900 c[0] + 6700 c[1] + 7900 c[2]    250000
pest_s : -0.8 s[0] + 0.19999999999999996 s[1] + 1.2 s[2]    0
pest_w : -0.7 w[0] + 0.30000000000000004 w[1] + 1.3 w[2]    0
pest_c : -0.6 c[0] + 0.4 c[1] + 1.4 c[2]    0
s[0]    0
s[1]    0
s[2]    0
w[0]    0
w[1]    0
w[2]    0
c[0]    0
c[1]    0
c[2]    0

```

```

optimize!(crop_model)

println("Optimized soybean values:")
@show value.(s[i] for i in 0:2);

println("Optimized wheat values:")
@show value.(w[i] for i in 0:2);

println("Optimized corn values:")
@show value.(c[i] for i in 0:2);

println("Optimized objective value:")
@show objective_value(crop_model);

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms

LP has 7 rows; 9 cols; 27 nonzeros

Coefficient ranges:

Matrix [2e-01, 8e+03]

Cost [7e+02, 1e+03]

Bound [0e+00, 0e+00]

RHS [1e+02, 2e+05]

Presolving model

7 rows, 9 cols, 27 nonzeros 0s

7 rows, 9 cols, 27 nonzeros 0s

Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced

Problem not reduced by presolve: solving the LP

Using EKK dual simplex solver - serial

```

Iteration      Objective      Infeasibilities num(sum)
      0    -1.8170297740e+02 Ph1: 7(28.3601); Du: 9(181.703) 0s
      7      1.1674116702e+05 Pr: 0(0) 0s
Model status      : Optimal
Simplex iterations: 7
Objective value    : 1.1674116702e+05
P-D objective error : 0.0000000000e+00
HiGHS run time     : 0.00
Optimized soybean values:
value.((s[i] for i = 0:2)) = [13.812154696132604, 55.24861878453038, 0.0]
Optimized wheat values:
value.((w[i] for i = 0:2)) = [6.743306417339563, 15.734381640458977, 0.0]
Optimized corn values:
value.((c[i] for i = 0:2)) = [26.92307692307692, 0.0, 11.538461538461547]
Optimized objective value:
objective_value(crop_model) = 116741.16702082448

```

```

println("Shadow values:")
@show shadow_price(land);

@show shadow_price(ds);
@show shadow_price(dw);
@show shadow_price(dc);

@show shadow_price(pest_w);
@show shadow_price(pest_w);
@show shadow_price(pest_c);

```

```

Shadow values:
shadow_price(land) = 729.4
shadow_price(ds) = 0.046353591160220996
shadow_price(dw) = -0.0
shadow_price(dc) = 0.04132307692307693
shadow_price(pest_w) = 91.99999999999993
shadow_price(pest_w) = 91.99999999999993
shadow_price(pest_c) = 108.67692307692309

```

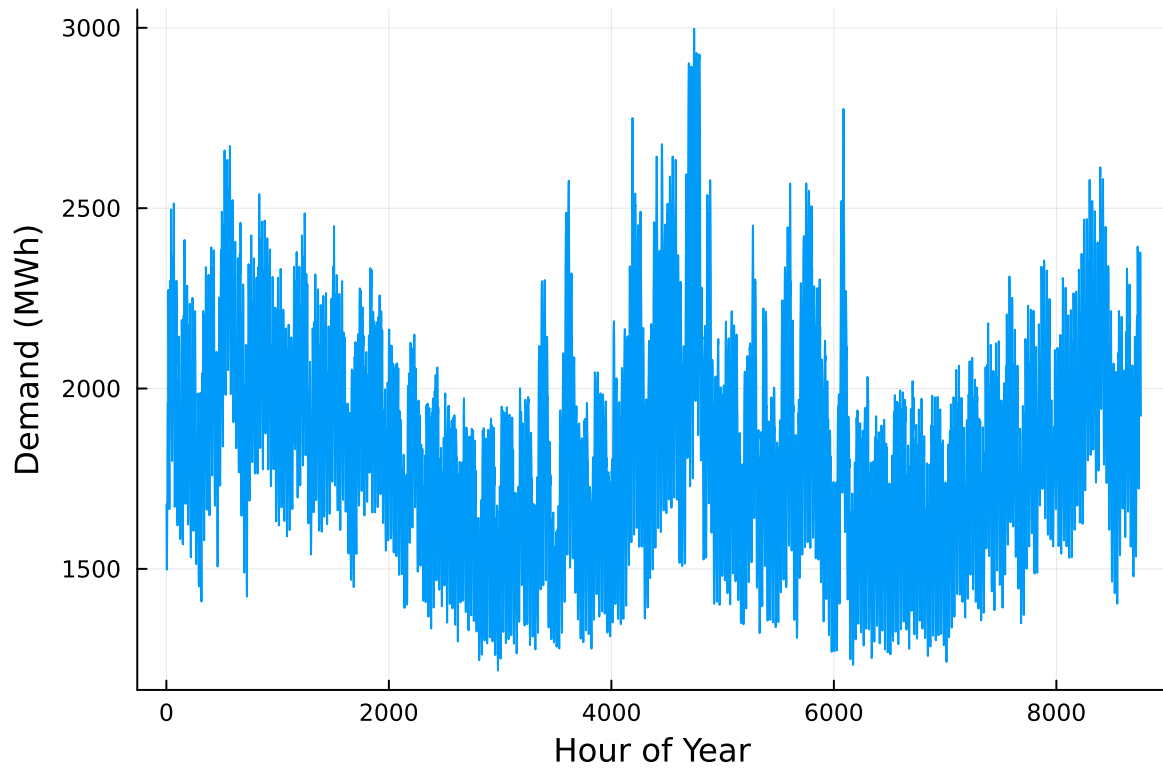
```

# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, "Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)

```

```
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs (\$/MW installed), variable costs (\$/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```
# load capacity factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO₂ limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO₂/yr.

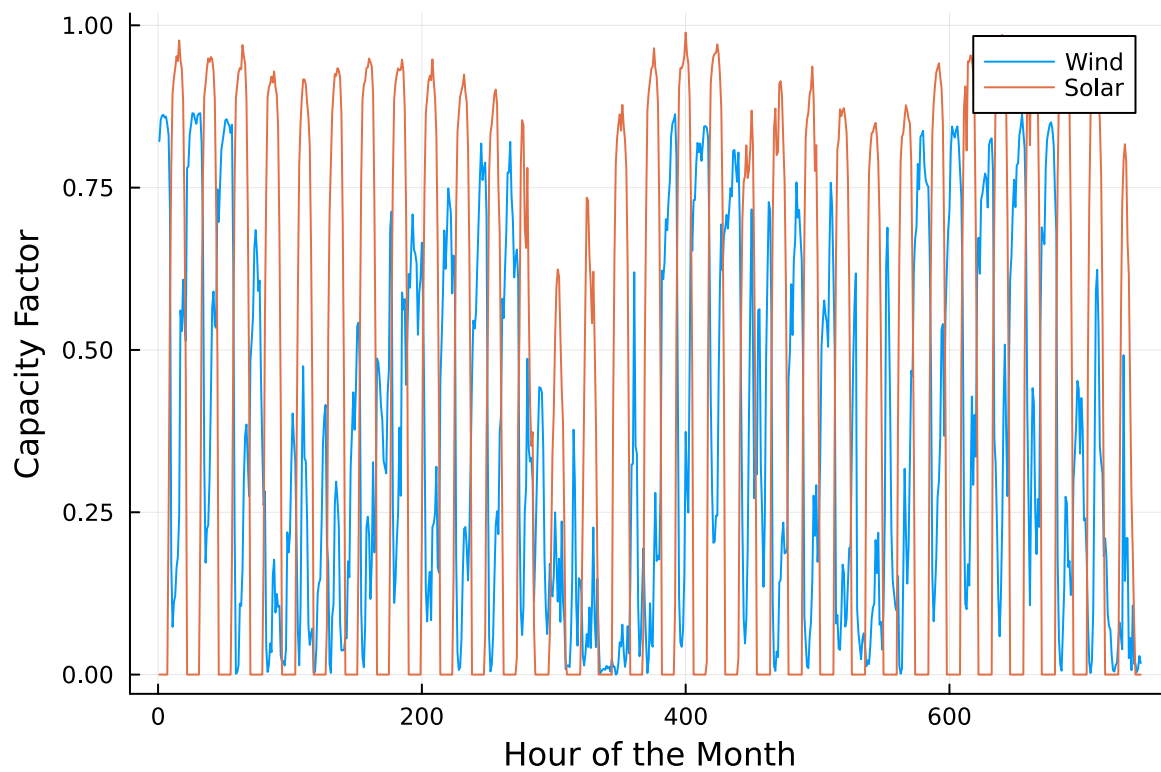
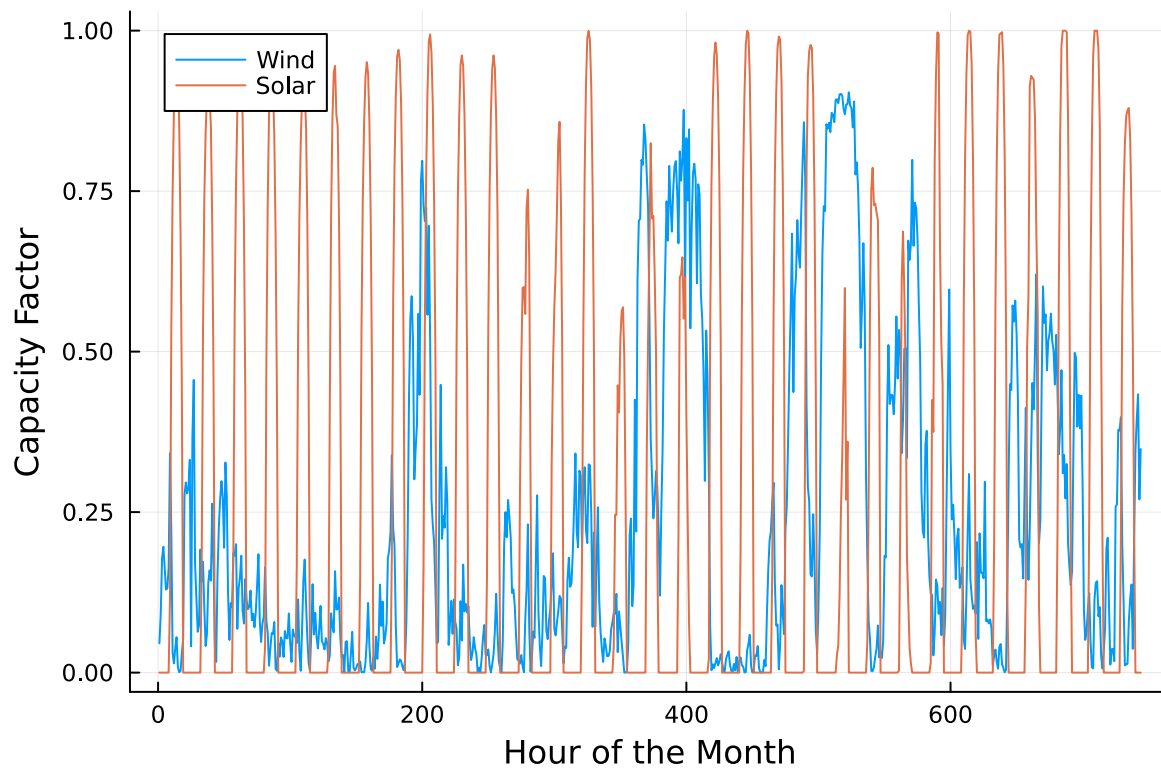
While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of \$10,000/MWh.

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?



Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO₂? What is the value to the utility be of allowing it to emit an additional 1000 tCO₂/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a JuMP variable `x`, but it will return a special container which holds other information about `x` that is useful for JuMP. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

References

List any external references consulted, including classmates.